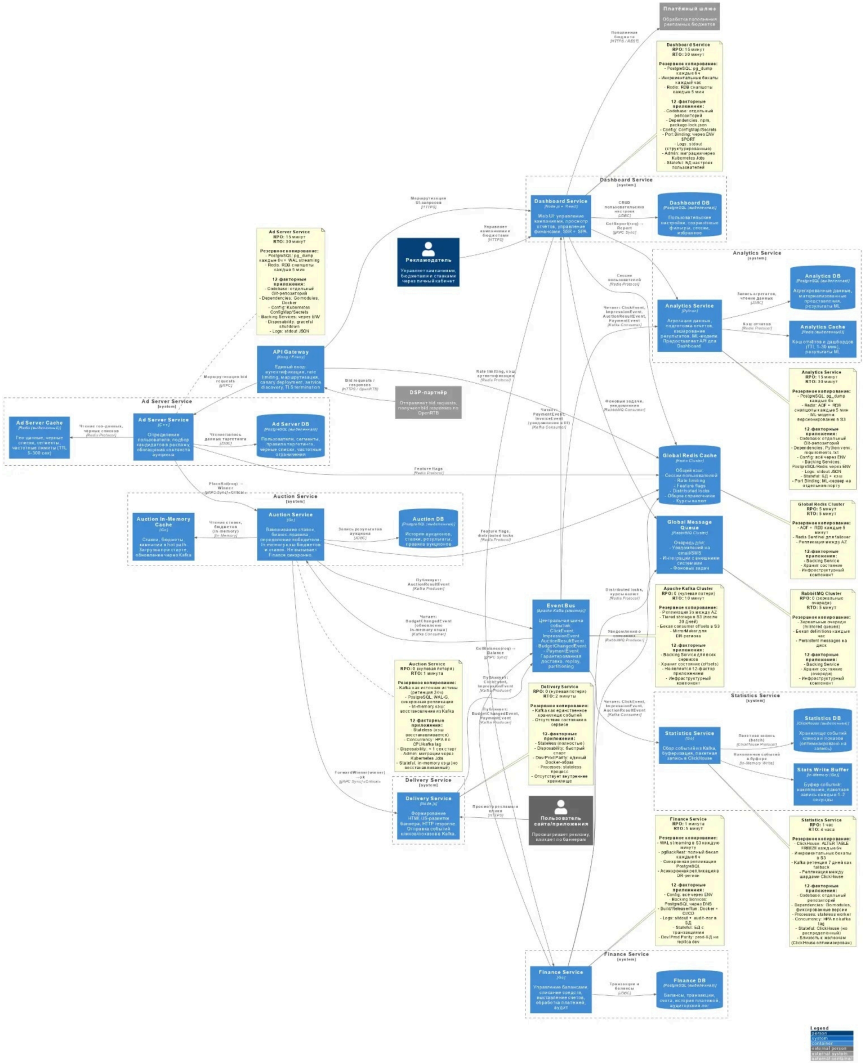
C4 Container Diagram — AdScale (TO BE Year+1: Микросервисная архитектура)


Платёжный шлюз
Обработка пополнения рекламных бюджетов
Пополнение бюджета [HTTPS / REST]
Dashboard Service [system]
Dashboard Service [Node.js + React]
Web UI: управление кампаниями, просмотр отчётов, управление финансами. SSR + SPA.
CRUD пользовательских настроек [JDBC]
GetReport(req) → Report [gRPC Sync]
Dashboard DB [PostgreSQL (выделенная)]
Пользовательские настройки, сохранённые фильтры, сессии, избранное
Маршрутизация UI-запросов [HTTPS]
Рекламодатель
Управляет кампаниями, бюджетами и ставками через личный кабинет
Управляет кампаниями и бюджетами [HTTPS]
Сессии пользователей [Redis Protocol]
Читает: ClickEvent, ImpressionEvent, AuctionResultEvent, PaymentEvent [Kafka Consumer]
Analytics Service [system]
Analytics Service [Python]
Агрегация данных, подготовка отчетов, кэширование результатов, ML-модели. Предоставляет API для Dashboard
Запись агрегатов, чтение данных [JDBC]
Analytics DB [PostgreSQL (выделенная)]
Агрегированные данные, материализованные представления, результаты ML
Кэш отчетов [Redis Protocol]
Analytics Cache [Redis (выделенный)]
Кэш отчётов и дашбордов (TTL 1-30 мин), результаты ML
API Gateway [Kong / Envoy]
Единый вход: аутентификация, rate limiting, маршрутизация, canary deployment, service discovery, TLS termination
Маршрутизация bid requests [gRPC]
Bid requests / responses [HTTPS / OpenRTB]
DSP-партнёр
Отправляет bid requests, получает bid responses по OpenRTB
Rate limiting, кэш аутентификации [Redis Protocol]
Ad Server Service [system]
Ad Server Cache [Redis (выделенный)]
Гео-данные, черные списки, сегменты, частотные лимиты (TTL 5-300 сек)
Чтение гео-данных, черных списков [Redis Protocol]
Ad Server Service [C++]
Определение пользователя, подбор кандидатов в рекламу, обогащение контекста аукциона
Чтение/запись данных таргетинга [JDBC]
Ad Server DB [PostgreSQL (выделенная)]
Пользователи, сегменты, правила таргетинга, черные списки, частотные ограничения
Feature flags [Redis Protocol]
Читает: PaymentEvent, InvoiceEvent (уведомления в UI) [Kafka Consumer]
Фоновые задачи, уведомления [RabbitMQ Consumer]
Global Redis Cache [Redis Cluster]
Общий кэш: - Сессии пользователей - Rate limiting - Feature flags - Distributed locks - Общие справочники - Курсы валют
PlaceBid(req) → Winner [gRPC Sync] «Critical»
Auction Service [system]
Auction In-Memory Cache [Go]
Ставки, бюджеты, кампании в hot path. Загрузка при старте, обновление через Kafka
Чтение ставок, бюджетов (in-memory) [In-Memory]
Auction Service [Go]
Взвешивание ставок, бизнес-правила, определение победителя. In-memory кэш бюджетов и ставок. Не вызывает Finance синхронно.
Запись результатов аукциона [JDBC]
Auction DB [PostgreSQL (выделенная)]
История аукционов, ставки, результаты, правила аукционов
Feature flags, distributed locks [Redis Protocol]
Global Message Queue [RabbitMQ Cluster]
Очередь для: - Уведомлений на email/SMS - Интеграции с внешними системами - Фоновых задач
Публикует: AuctionResultEvent [Kafka Producer]
Читает: BudgetChangedEvent (обновление in-memory кэша) [Kafka Consumer]
Distributed locks, курсы валют [Redis Protocol]
Event Bus [Apache Kafka (кластер)]
Центральная шина событий. - ClickEvent, ImpressionEvent - AuctionResultEvent - BudgetChangedEvent - PaymentEvent Гарантированная доставка, replay, partitioning
Уведомления о списаниях [RabbitMQ Producer]
GetBalance(req) → Balance [gRPC Sync]
Публикует: ClickEvent, ImpressionEvent [Kafka Producer]
Публикует: BudgetChangedEvent, PaymentEvent [Kafka Producer]
Читает: ClickEvent, ImpressionEvent, AuctionResultEvent [Kafka Consumer]
Statistics Service [system]
Statistics Service [Go]
Сбор событий из Kafka, буферизация, пакетная запись в ClickHouse
Пакетная запись (batch) [ClickHouse Protocol]
Statistics DB [ClickHouse (выделенная)]
Хранилище событий кликов и показов (оптимизировано на запись)
Накопление событий в буфере [In-Memory Write]
Stats Write Buffer [In-Memory (Go)]
Буфер событий: накопление, пакетная запись каждые 1-2 секунды
ForwardWinner(winner) → ok [gRPC Sync] «Critical»
Delivery Service [system]
Delivery Service [Node.js]
Формирование HTML/JS-разметки баннера, HTTP response. Отправка событий кликов/показов в Kafka.
Просмотр рекламы и клики [HTTPS]
Пользователь сайта/приложения
Просматривает рекламу, кликает по баннерам
Finance Service [system]
Finance Service [Go]
Управление балансами, списание средств, выставление счетов, обработка платежей, аудит
Транзакции и балансы [JDBC]
Finance DB [PostgreSQL (выделенная)]
Балансы, транзакции, счета, история платежей, аудиторский лог
Legend
person
system
container
external person
external system
external container

Dashboard Service
RPO: 15 минут
RTO: 30 минут

Резервное копирование:
- PostgreSQL: pg_dump каждые 6ч
- Инкрементальные бекапы каждый час
- Redis: RDB снапшоты каждые 5 мин

12-факторные приложения:
- Codebase: отдельный репозиторий
- Dependencies: npm, package-lock.json
- Config: ConfigMap/Secrets
- Port Binding: через ENV $PORT
- Logs: stdout (структурированные)
- Admin: миграции через Kubernetes Jobs
- Stateful: БД настроек пользователей

Ad Server Service
RPO: 15 минут
RTO: 30 минут

Резервное копирование:
- PostgreSQL: pg_dump каждые 6ч + WAL streaming
- Redis: RDB снапшоты каждые 5 мин

12-факторные приложения:
- Codebase: отдельный Git-репозиторий
- Dependencies: Go modules, Docker
- Config: Kubernetes ConfigMap/Secrets
- Backing Services: через ENV
- Disposability: graceful shutdown
- Logs: stdout JSON

Analytics Service
RPO: 15 минут
RTO: 30 минут

Резервное копирование:
- PostgreSQL: pg_dump каждые 6ч
- Redis: AOF + RDB снапшоты каждые 5 мин
- ML-модели: версионирование в S3

12-факторные приложения:
- Codebase: отдельный Git-репозиторий
- Dependencies: Python venv, requirements.txt
- Config: всё через ENV
- Backing Services: PostgreSQL/Redis через ENV
- Logs: stdout JSON
- Stateful: БД + кэш
- Port Binding: ML-сервер на отдельном порту

Global Redis Cluster
RPO: 5 минут
RTO: 5 минут

Резервное копирование:
- AOF + RDB каждые 5 минут
- Redis Sentinel для failover
- Репликация между AZ

12-факторные приложения:
- Backing Service
- Хранит состояние
- Инфраструктурный компонент

Apache Kafka Cluster
RPO: 0 (нулевая потеря)
RTO: 10 минут

Резервное копирование:
- Репликация 3x между AZ
- Tiered storage в S3 (после 30 дней)
- Бекап consumer offsets в S3
- MirrorMaker для DR-региона

12-факторные приложения:
- Backing Service для всех сервисов
- Хранит состояние (offsets)
- Не является 12-фактор приложением
- Инфраструктурный компонент

RabbitMQ Cluster
RPO: 0 (зеркальные очереди)
RTO: 5 минут

Резервное копирование:
- Зеркальные очереди (mirrored queues)
- Бекап definitions каждые час
- Persistent messages на диск

12-факторные приложения:
- Backing Service
- Хранит состояние (очереди)
- Инфраструктурный компонент

Auction Service
RPO: 0 (нулевая потеря)
RTO: 1 минута

Резервное копирование:
- Kafka как источник истины (ретенция 24ч)
- PostgreSQL: WAL-G, синхронная репликация
- In-memory кэш: восстановление из Kafka

12-факторные приложения:
- Stateless (кэш восстанавливается)
- Concurrency: HPA по CPU/kafka lag
- Disposability: < 1 сек старт
- Admin: миграции через Kubernetes Jobs
- Stateful: in-memory кэш (не восстанавливаемый)

Delivery Service
RPO: 0 (нулевая потеря)
RTO: 2 минуты

Резервное копирование:
- Kafka как единственное хранилище событий
- Отсутствие состояния в сервисе

12-факторные приложения:
- Stateless (полностью)
- Disposability: быстрый старт
- Dev/Prod Parity: единый Docker-образ
- Processes: stateless процесс
- Отсутствует внутреннее хранилище

Finance Service
RPO: 1 минута
RTO: 5 минут

Резервное копирование:
- WAL streaming в S3 каждую минуту
- pgBackRest: полный бекап каждые 6ч
- Синхронная репликация PostgreSQL
- Асинхронная репликация в DR-регион

12-факторные приложения:
- Config: всё через ENV
- Backing Services: PostgreSQL через DNS
- Build/Release/Run: Docker + CI/CD
- Logs: stdout + audit-лог в БД
- Stateful: БД с транзакциями
- Dev/Prod Parity: prod-БД не replica dev

Statistics Service
RPO: 1 час
RTO: 4 часа

Резервное копирование:
- ClickHouse: ALTER TABLE FREEZE каждые 6ч
- Инкрементальные бекапы в S3
- Kafka ретенция 7 дней как fallback
- Репликация между шардами ClickHouse

12-факторные приложения:
- Codebase: отдельный репозиторий
- Dependencies: Go modules, фиксированные версии
- Processes: stateless worker
- Concurrency: HPA по kafka lag
- Stateful: ClickHouse (но распределенный)
- Близость к железкам (ClickHouse оптимизирован)